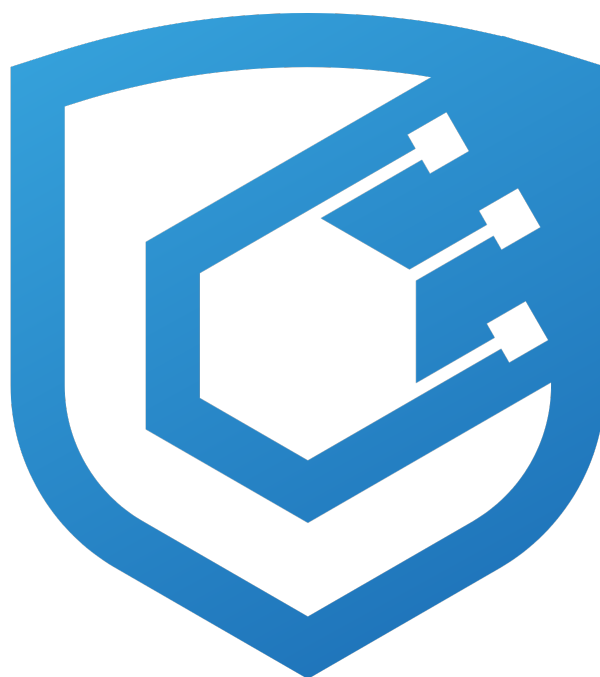




Securitize Jump Router Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Farouk](#)

[JesJupyter](#)

[Ctrus](#)

June 9, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	3
7	Findings	6
7.1	Low Risk	6
7.1.1	ExecuteSwap::swap_asset_for_liquidity charges the sell-path fee on JUMP output instead of the QUODD-referenced input	6
7.1.2	MbpsFeeManager::validate performs no external fee collector zero-address rejection required by the design specification	6
7.1.3	Protocol fee is charged on the gross amount instead of the post-external-fee remainder	7
7.1.4	Rounding bias in calculate_jump_price() skews max_positive_slippage_mbps enforcement across swap directions	8
7.1.5	Swap price events are denormalized using asset decimals instead of liquidity decimals	9
7.2	Informational	11
7.2.1	calculate_jump_price requires mint decimals <= 18 but initialize and update_jump_pool_config never validate the bound	11
7.2.2	JumpRouterState collar and min-amount parameters have no bounds at init or in their setters	11
7.2.3	UpdateJumpPoolConfig::update_jump_pool_config repoints mints but does not reset min_asset_amount_in, min_liquidity_amount_in	12
7.2.4	calculate_jump_price returns a liquidity-over-asset ratio while three unit tests assert the inverse	12
7.2.5	JumpRouterCounter uses init_if_needed with a fixed singleton seed	13
7.2.6	MbpsFeeManager doc comments mislabel milli-basis-points as basis points	13
7.2.7	Source filename mpbs_fee_manager.rs has transposed letters	13
7.2.8	Standard execute_swap accepts an empty regulatory external_id	14
7.2.9	RedemptionCompleted emits gross liquidity output rather than the user's net received amount	14
7.2.10	Use a two-step admin transfer flow to reduce the risk of irreversible governance mistakes	15
7.2.11	Router admin can repoint an existing router to an unrelated Jump pool	16
7.2.12	Fee collector default pubkey is not rejected while Initialization	16
7.2.13	Combined internal and external fees are not capped	17
7.3	Gas Optimization	18
7.3.1	Recomputed power-of-ten divisor in repeated per-swap price denormalization	18

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Securitize SVM Jump Router is an on-chain swap-routing program for the Solana Virtual Machine that intermediates trades between regulated digital security tokens (the “asset” mint) and a paired liquidity token (the “liquidity” mint, typically a stablecoin), executing the underlying trade against a Jump Prop AMM pool. Rather than holding liquidity itself, the router sits between the end user and the external AMM, enforcing the protocol’s own pricing collar, fee policy, and access controls on top of every swap that flows through the Jump pool.

A swap proceeds in two directions: liquidity-for-asset (an on-ramp into the security token) and asset-for-liquidity (an off-ramp back into the liquidity token). In both cases the router pulls the input tokens from the user into a router-owned vault, requests a quote from the Jump pool, derives an implied execution price (`calculate_jump_price`) normalized to a fixed WAD scale, and compares that price against an operator-supplied NBBO reference. If the realized price improves on the reference by more than the configured `max_positive_slippage_mbps` collar, the swap is rejected; otherwise the router signs a CPI into the Jump program using a program-derived authority, verifies that the amount received exactly matches the quoted amount, deducts protocol and optional external fees from the liquidity leg, and forwards the net proceeds to the user. Fees are computed in milli-basis-points against a 100,000 denominator and routed to a designated fee-collector token account.

The system is bounded by several explicit rules: fee numerators may not exceed a 50% cap, mint decimals must not exceed the WAD scale of 18, per-direction minimum input amounts gate dust trades, quote requests must carry an unexpired `expires_at` deadline, and the realized amount received from the pool must equal the quoted amount or the transaction reverts. Two execution entry points exist — an operator-gated `execute_swap` that carries the NBBO price and slippage collar, and a `execute_swap_headless` path that omits the operator price reference — each independently pausable through bitflag pause controls.

Architecturally, the program centers on a `JumpRouterState` account that stores the admin, operator set, fee manager, Jump pool configuration, slippage collar, and minimum-amount parameters, all seeded as a PDA alongside a separate authority PDA that signs outbound CPIs and owns the router’s intermediate vaults. A `JumpPoolConfig` records the bound pool address and the asset/liquidity mint pairing and side, validated against the live Jump pool state at registration time. Token movement is abstracted through a `transfer_tokens!` macro built on SPL `TransferChecked`, and the fee logic is encapsulated in a milli-basis-points fee manager. Supporting components include administrative instructions for rotating the admin, managing operators, repointing the pool configuration,

adjusting fees and collar parameters, and toggling pause flags, together with a rich set of emitted events (Swap, RedemptionCompleted, PstRamp) intended for off-chain accounting and regulatory reporting.

For the system to be safe, the price derived from the Jump quote and the NBBO collar must correctly bound execution across both swap directions, fees must be deducted accurately and never exceed their caps, and the router must only ever release the exact tokens it received from the pool — with every privileged configuration change leaving the program's vaults and accounting in a consistent state.

5 Audit Scope

The audit scope was limited to:

```
programs/bc-solana-jump-router-sc/src/lib.rs
programs/bc-solana-jump-router-sc/src/constants.rs
programs/bc-solana-jump-router-sc/src/errors.rs
programs/bc-solana-jump-router-sc/src/events.rs
programs/bc-solana-jump-router-sc/src/utils/common.rs
programs/bc-solana-jump-router-sc/src/utils/get_authority_signer.rs
programs/bc-solana-jump-router-sc/src/utils/swap_utils.rs
programs/bc-solana-jump-router-sc/src/utils/transfer_tokens.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/swap.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/constants.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/request_for_quote.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/jump_pool_state.rs
programs/bc-solana-jump-router-sc/src/states/jump_router_counter.rs
programs/bc-solana-jump-router-sc/src/states/jump_pool_config.rs
programs/bc-solana-jump-router-sc/src/states/jump_router_state.rs
programs/bc-solana-jump-router-sc/src/states/fee_manager/fee_manager_core.rs
programs/bc-solana-jump-router-sc/src/states/fee_manager/mpbs_fee_manager.rs
programs/bc-solana-jump-router-sc/src/instructions/swap_accounts.rs
programs/bc-solana-jump-router-sc/src/instructions/operator/execute_swap.rs
programs/bc-solana-jump-router-sc/src/instructions/user/execute_swap_headless.rs
programs/bc-solana-jump-router-sc/src/instructions/cpi/prop_amm/swap.rs
programs/bc-solana-jump-router-sc/src/instructions/cpi/prop_amm/request_for_quote.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/initialize.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/update_pause_flags.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/add_operator.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/revoke_operator.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/change_admin.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/update_fee_manager.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/update_jump_pool_config.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/update_max_positive_slippage.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/update_min_asset_amount_in.rs
programs/bc-solana-jump-router-sc/src/instructions/admin/update_min_liquidity_amount_in.rs
```

6 Executive Summary

Over the course of 6 days, the Cyfrin team conducted an audit on the [Jump Router](#) code provided by [Securitize](#). The findings consist of 5 Low severity issues with the remainder being informational and gas optimizations.

This review examined the Securitize SVM Jump Router, an Anchor-based Solana program that routes swaps between a regulated digital security token and a paired liquidity token through an external Jump Prop AMM pool. The security model rests on the router faithfully reproducing the pool's economics while layering its own pricing collar, fee deductions, and access controls: it depends on correct WAD-scale price normalization, accurate fee accounting on the liquidity leg, a slippage collar that is enforced consistently across both swap directions, and reliable settlement of the token amounts quoted by the Jump pool.

The review concentrated on the swap execution paths and their interaction with the external pool, the fee-manager arithmetic and its interplay with the optional external fee, the price-collar and NBBO logic across both buy and sell directions, the validation performed when mints and the Jump pool are registered or repointed, and the program's administrative and operator privilege surface.

The most consequential accepted findings centered on fee-accounting semantics, price-collar consistency, and event/reporting accuracy. The fee findings identified cases where the sell path charges fees on the realized Jump output rather than the QUODD/NBBO-referenced value, where the protocol fee is computed from the gross amount instead of the post-external-fee remainder, and where individually valid internal and external fee managers can exceed the intended aggregate fee limit when combined. The client acknowledged these fee-model differences where they were intentional or documentation-driven.

The pricing and accounting findings focused on the positive-slippage guard and off-chain observability. The router originally used a rounded scalar `calculate_jump_price()` value for both swap directions, which made `max_positive_slippage_mbps` enforcement sensitive to rounding direction; this was resolved by checking the exact rational price and using direction-specific conservative rounding for emitted values. The report also identified event-denormalization and net-versus-gross event-value issues, along with missing or inconsistent validation around mint decimals, zero fee collectors, regulatory `external_id` values, and stale minimum thresholds after pool reconfiguration. The development team was responsive throughout, resolving the majority of the accepted issues during the engagement and acknowledging the remaining operational or documentation-driven risks.

Centralization Risks

The protocol is, by the regulated nature of its asset, highly trust-dependent on its administrators. The router admin can rotate the admin key, add and revoke operators, repoint the Jump pool configuration to a different pool and mint pairing, replace the fee manager and fee collector, adjust the slippage collar and minimum-amount parameters, and pause either swap path. Operators hold meaningful discretion as well, supplying the NBBO reference price that drives the slippage collar on the primary swap path. Because the program is non-upgradeable, administrators cannot patch a latent bug in place once deployed — which raises the importance of correct initial deployment — but the configuration surface above still requires users to place substantial trust in the admin and operator set when transacting through the router.

Summary

Project Name	Jump Router
Repository	bc-bd-router-sc
Commit	bf11b73be582...
Audit Timeline	June 2nd - June 9th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	5
Informational	13
Gas Optimizations	1
Total Issues	19

Summary of Findings

[L-1] <code>ExecuteSwap::swap_asset_for_liquidity</code> charges the sell-path fee on JUMP output instead of the QUODD-referenced input	Acknowledged
[L-2] <code>MbpsFeeManager::validate</code> performs no external fee collector zero-address rejection required by the design specification	Resolved
[L-3] Protocol fee is charged on the gross amount instead of the post-external-fee remainder	Acknowledged
[L-4] Rounding bias in <code>calculate_jump_price()</code> skews <code>max_positive_slip_page_mbps</code> enforcement across swap directions	Resolved
[L-5] Swap price events are denormalized using asset decimals instead of liquidity decimals	Acknowledged
[I-01] <code>calculate_jump_price</code> requires mint decimals ≤ 18 but <code>initialize</code> and <code>update_jump_pool_config</code> never validate the bound	Resolved
[I-02] <code>JumpRouterState</code> collar and min-amount parameters have no bounds at <code>init</code> or in their setters	Acknowledged
[I-03] <code>UpdateJumpPoolConfig::update_jump_pool_config</code> repoints mints but does not reset <code>min_asset_amount_in</code> , <code>min_liquidity_amount_in</code>	Resolved
[I-04] <code>calculate_jump_price</code> returns a liquidity-over-asset ratio while three unit tests assert the inverse	Resolved
[I-05] <code>JumpRouterCounter</code> uses <code>init_if_needed</code> with a fixed singleton seed	Resolved
[I-06] <code>MbpsFeeManager</code> doc comments mislabel milli-basis-points as basis points	Resolved
[I-07] Source filename <code>mpbs_fee_manager.rs</code> has transposed letters	Resolved
[I-08] Standard <code>execute_swap</code> accepts an empty regulatory <code>external_id</code>	Resolved
[I-09] <code>RedemptionCompleted</code> emits gross liquidity output rather than the user's net received amount	Resolved
[I-10] Use a two-step admin transfer flow to reduce the risk of irreversible governance mistakes	Acknowledged
[I-11] Router admin can repoint an existing router to an unrelated Jump pool	Acknowledged
[I-12] Fee collector default pubkey is not rejected while Initialization	Resolved
[I-13] Combined internal and external fees are not capped	Acknowledged
[G-1] Recomputed power-of-ten divisor in repeated per-swap price denormalization	Acknowledged

7 Findings

7.1 Low Risk

7.1.1 `ExecuteSwap::swap_asset_for_liquidity` charges the sell-path fee on JUMP output instead of the QUODD-referenced input

Description: The sell path computes both liquidity-denominated fees from JUMP's realized output rather than from the QUODD/NBBO-referenced input value.

In the current shared swap implementation, `swap_asset_for_liquidity` obtains the JUMP quote and then calculates the protocol fee from `quoted_liquidity_amount`:

```
let liquidity_fee_amount = jump_router_state
    .fee_manager
    .calculate_fee(quoted_liquidity_amount)?;
```

It applies the same output-based fee base to any external fee manager:

```
let liquidity_external_fee_amount = match external_fee_manager {
    Some(manager) => manager.calculate_fee(quoted_liquidity_amount)?,
    None => 0,
};
```

These lines are in `programs/bc-solana-jump-router-sc/src/instructions/swap_accounts.rs:466-490`.

`quoted_liquidity_amount` is JUMP's actual USDC quote, not the QUODD/NBBO reference value. On the standard operator-gated path, `nbbo_price` is supplied by `execute_swap_handler` and passed into `SwapAccounts::execute_swap` (`programs/bc-solana-jump-router-sc/src/instructions/operator/execute_swap.rs:20-55`). On the headless path, no NBBO value is available; `execute_swap_headless_handler` intentionally passes `None` for `nbbo_price` (`programs/bc-solana-jump-router-sc/src/instructions/user/execute_swap_headless.rs:29-37`).

The protocol specification describes the sell-side Securitize fee as a fixed fee on the input token's USDC-equivalent value at the QUODD/NBBO reference. Under the current implementation, if JUMP delivers more liquidity than the NBBO-equivalent amount, the protocol fee and external fee are also charged on the positive execution improvement. The buy path is different: it computes fees from `amount_in` before sending the post-fee amount to JUMP (`swap_accounts.rs:270-278`).

Impact: For standard operator-gated sell swaps where JUMP's execution beats the QUODD/NBBO reference, users pay `fee_bps` on the positive execution improvement instead of receiving that improvement in full. The overcharge is bounded by the configured positive-slippage collar on the standard path. The current headless path has no NBBO input, so the NBBO-referenced sell-fee model cannot be implemented for headless swaps without changing either the API or the documented headless behavior.

Recommended Mitigation: For standard `execute_swap`, compute the sell-path protocol fee from the NBBO-equivalent liquidity value rather than `quoted_liquidity_amount`. If the external fee is intended to follow the same basis, compute it from the same reference amount. If the current output-based model is intended, update the specification to state that sell fees are charged on realized JUMP output.

For `execute_swap_headless`, either document that sell fees are output-based because no NBBO is supplied, or add a headless NBBO/reference input if headless swaps must also use the QUODD-referenced fee model.

Securitize: Acknowledged; I believe this is a doc error, and we intentionally deduct fee from the real swap amount.

7.1.2 `MbpsFeeManager::validate` performs no external fee collector zero-address rejection required by the design specification

Description: The design specification requires the program to reject a zero external fee collector when an external fee is non-zero. The current external-fee validation checks the caller-supplied external manager and associated token account, but it does not reject `Pubkey::default()`.

SwapAccounts::validate_external_fee_config validates that:

- the external manager passes `external_fee_manager.validate()`;
- `external_fee_manager.fee_collector_wallet()` equals the supplied `external_fee_collector_wallet`;
- `external_fee_collector_ata` is the expected ATA for that wallet, the liquidity mint, and the liquidity token program;
- the ATA mint, owner, and account owner match the expected values.

Those checks are at `programs/bc-solana-jump-router-sc/src/instructions/swap_accounts.rs:171-212`.

MbpsFeeManager::validate only enforces the fee numerator cap (`programs/bc-solana-jump-router-sc/src/states/fee_manager/mpbs_fee_manager.rs:42-48`). It does not inspect `collector_wallet`. Therefore an external manager with a non-zero numerator and `collector_wallet = Pubkey::default()` is not rejected by the fee-manager validation itself. If the caller supplies the corresponding zero-authority ATA, the external fee can be routed to an address with no recoverable private signer.

This is no longer the same issue as the earlier stale `active_fee_manager` wording: the protocol fee collector is still validated against `jump_router_state.fee_manager.fee_collector_wallet()` at `swap_accounts.rs:111-115`, and protocol fees are still transferred to `fee_collector_ata`. The missing guard is specific to the additional external fee recipient.

Impact: When an external fee manager with a non-zero numerator uses the default public key as the collector, the external fee may be transferred to an unrecoverable zero-authority ATA instead of a valid integrator collector. The stored Securitize protocol fee is unaffected because it is collected through the configured protocol fee collector.

Recommended Mitigation: Reject `Pubkey::default()` in the external-fee validation path before any fund movement. At minimum, enforce this when `external_fee_manager` has a non-zero numerator. A stricter and simpler approach is to reject a default external collector whenever `external_fee_manager.is_some()`.

Securitize: Fixed in [64086a4](#).

Cyfrin: Verified.

7.1.3 Protocol fee is charged on the gross amount instead of the post-external-fee remainder

Description: FR-2 specifies that when both the stored Securitize fee and an optional external fee apply to a swap, the external fee is deducted first and the Securitize fee is then applied only to the remaining amount. The current implementation deducts both fees, but computes each one from the same gross input rather than charging the Securitize fee on the post-external-fee remainder.

This finding's primary proof is the buy path, where both fees are calculated from `amount_in`:

```
let liquidity_fee_amount = jump_router_state.fee_manager.calculate_fee(amount_in)?;
let liquidity_external_fee_amount = match external_fee_manager {
    Some(manager) => manager.calculate_fee(amount_in)?,
    None => 0,
};
let liquidity_swap_amount = amount_in
    .checked_sub(liquidity_fee_amount)
    .and_then(|x| x.checked_sub(liquidity_external_fee_amount))
    .ok_or(JumpRouterError::MathOverflow)?;
```

The same gross-before-external ordering also appears in the current sell implementation, where both fees are computed from the full `quoted_liquidity_amount`:

```
let liquidity_fee_amount = jump_router_state
    .fee_manager
    .calculate_fee(quoted_liquidity_amount)?;

let liquidity_external_fee_amount = match external_fee_manager {
    Some(manager) => manager.calculate_fee(quoted_liquidity_amount)?,
    None => 0,
```



```
};
```

The spec-conformant sequence is:

1. `external_fee_amount = external_rate * gross`
2. `securitize_fee_amount = securitize_rate * (gross - external_fee_amount)`
3. `jump_input = gross - external_fee_amount - securitize_fee_amount`

The implemented sequence computes the Securitize fee on `gross` rather than on `gross - external_fee_amount`, so the Securitize fee is slightly larger than FR-2 intends whenever an external fee is present.

Impact: The effect is a deterministic overcollection of the Securitize fee on external-fee swaps, equal to `securitize_rate * external_fee_amount`. At realistic fee rates this is a small, second-order amount — for example, a 1% Securitize fee and a 1% external fee on a gross of 1,000,000 overcharges by roughly 100 units ($0.01 * 10,000$), i.e. about 0.01% of notional. The magnitude only becomes material if both fee rates are configured near the 50% per-fee cap, since each manager is validated independently against that cap.

Recommendation:

Apply FR-2 literally on both swap paths: compute the external fee first, subtract it, then compute the stored Securitize fee on the remainder.

Securitize: Acknowledged; This is a doc bug.

7.1.4 Rounding bias in `calculate_jump_price()` skews `max_positive_slippage_mbps` enforcement across swap directions

Description: The `max_positive_slippage_mbps` protection relies on `calculate_jump_price()`, which always rounds the computed price down via integer division. Because the protection compares `jump_price` against `nbbo_price` in opposite directions for the two swap flows, using the same rounding direction in both cases introduces inconsistent enforcement across swap directions.

`calculate_jump_price()` derives a WAD-scaled price by dividing two normalized integers:

```
// Calculate price with WAD scale
let price = U256::from(normalized_liquidity_amount)
    .checked_mul(U256::from(10u128.pow(WAD_SCALE as u32)))
    .ok_or_else(|| error!(JumpRouterError::MathOverflow))?
    .checked_div(U256::from(normalized_asset_amount))
    .ok_or_else(|| error!(JumpRouterError::MathOverflow));
```

This uses integer division and therefore always floors the result.

The floored `jump_price` is then used for `max_positive_slippage_mbps` enforcement in both swap directions:

```
if let Some(nbbo_price) = nbbo_price_opt {
    if jump_price < *nbbo_price {
        let price_improvement_mbps =
            utils::swap_utils::calculate_price_improvement_mbps(&jump_price, nbbo_price)?;
        require!(
            price_improvement_mbps <= jump_router_state.max_positive_slippage_mbps,
            JumpRouterError::MaxPositiveSlippageExceeded
        );
    }
}
```

```
if let Some(nbbo_price) = nbbo_price_opt {
    if jump_price > *nbbo_price {
        let price_improvement_mbps =
            utils::swap_utils::calculate_price_improvement_mbps(nbbo_price, &jump_price)?;
        require!(
            price_improvement_mbps <= jump_router_state.max_positive_slippage_mbps,
```

```

        JumpRouterError::MaxPositiveSlippageExceeded
    );
}
}

```

This creates inconsistent behavior across swap directions:

- In the `LiquidityForAsset` path, the guard evaluates positive slippage when `jump_price < nbbo_price`, so flooring affects the comparison in one direction.
- In the `AssetForLiquidity` path, the guard evaluates positive slippage when `jump_price > nbbo_price`, so the same flooring affects the comparison in the opposite direction.

The issue does not affect the direct `min_amount_out_net` user slippage check, but it does undermine the correctness of the separate positive-slippage guard.

Impact: The configured `max_positive_slippage_mbps` limit is not enforced consistently across swap directions. This weakens the reliability of the control and can produce behavior that diverges from operator expectations and off-chain risk assumptions.

Recommended Mitigation: If keeping the current scalar-price approach, use direction-specific conservative rounding for the enforcement path.

Securitize: Fixed in [27e9c64](#) and [7757b81](#).

Cyfrin: Verified.

7.1.5 Swap price events are denormalized using asset decimals instead of liquidity decimals

Description: `calculate_jump_price` returns a WAD-scaled liquidity-per-asset price. In other words, it represents how many liquidity tokens are paid or received per one asset token.

However, when emitting swap events, the code denormalizes both `nbbo_price`, `jump_price`, and event rate using `asset_mint.decimals` instead of `liquidity_mint.decimals`.

```

utils::swap_utils::denormalize_price_to_decimals(
    &jump_price,
    swap_accounts.asset_mint.decimals,
)?;

```

Since the price is denominated in liquidity tokens, it should be scaled to the liquidity token's decimals.

Impact: On-chain swap execution and token settlement are not directly affected, but emitted accounting data can be incorrect.

Indexers or downstream integrations relying on `PstRamp.nbbo_price`, `PstRamp.jump_price`, `Swap.rate`, or `RedemptionCompleted.rate` may record materially incorrect prices.

Recommended Mitigation: Denormalize liquidity-per-asset prices using `liquidity_mint.decimals` instead of `asset_mint.decimals`:

```

let denormalized_nbbo_price = if let Some(nbbo_price) = nbbo_price_opt {
    utils::swap_utils::denormalize_price_to_decimals(
        nbbo_price,
        swap_accounts.liquidity_mint.decimals,
    )?
} else {
    0
};

let denormalized_jump_price = utils::swap_utils::denormalize_price_to_decimals(
    &jump_price,
    swap_accounts.liquidity_mint.decimals,
)?;

```

Securitize: Acknowledged; The indexer expects the price is scaled in asset decimals. It's also compatible with the on and off ramp programs.

7.2 Informational

7.2.1 `calculate_jump_price` requires mint decimals ≤ 18 but initialize and update_jump_pool_config never validate the bound

Description: `calculate_jump_price` opens with `require!(asset_decimals <= WAD_SCALE && liquidity_decimals <= WAD_SCALE, JumpRouterError::InvalidDecimals)` (programs/bc-solana-jump-router-sc/src/utils/swap_utils.rs:26-35). This check runs on every swap before collar and event-price calculations. Because Solana mint decimals are a u8 field, any configured mint with more than 18 decimals will make every swap revert with `InvalidDecimals`.

The mints are registered through the `Initialize` accounts and `initialize_handler` (programs/bc-solana-jump-router-sc/src/instructions/admin/initialize.rs:49-69, 120-127) and can be replaced through `update_jump_pool_config_handler` (programs/bc-solana-jump-router-sc/src/instructions/admin/update_jump_pool_config.rs:27-47, 88-99). Neither path checks `asset_mint.decimals` or `liquidity_mint.decimals` before storing the config.

Impact: If an asset or liquidity mint with decimals greater than 18 is registered, all swap instructions for that router instance will revert when price calculation runs. No trade can settle and no fees are collected until the config is corrected. If correction is not possible operationally, recovery requires creating or routing through another router instance.

Recommended Mitigation: Add decimals-bound checks in both `initialize_handler` and `update_jump_pool_config_handler`, requiring `asset_mint.decimals <= WAD_SCALE` and `liquidity_mint.decimals <= WAD_SCALE` before storing the pool config.

Securitize: Fixed in [297935b](#).

Cyfrin: Verified.

7.2.2 `JumpRouterState` collar and min-amount parameters have no bounds at init or in their setters

Description: Three `JumpRouterState` parameters are written without range validation.

First, `initialize_handler` stores `max_positive_slippage_mbps` verbatim (programs/bc-solana-jump-router-sc/src/instructions/admin/initialize.rs:120-145), and `update_max_positive_slippage_handler` only checks that the new value differs from the current one (programs/bc-solana-jump-router-sc/src/instructions/admin/update_max_positive_slippage.rs:20-32). No minimum or maximum is enforced.

The positive-slippage collar only applies when `nbbo_price_opt` is `Some`. That means it applies to standard operator-gated swaps, where `operator/execute_swap.rs` passes `Some(nbbo_price)` (programs/bc-solana-jump-router-sc/src/instructions/operator/execute_swap.rs:47-55), but not to headless swaps, where `execute_swap_headless_handler` passes `None` (programs/bc-solana-jump-router-sc/src/instructions/user/execute_swap_headless.rs:29-37). When the collar is active, the buy and sell checks use the stored value at `swap_accounts.rs:315-323` and `swap_accounts.rs:505-513`.

Second, `min_asset_amount_in` and `min_liquidity_amount_in` are updated by setters that only check `new != old` (programs/bc-solana-jump-router-sc/src/instructions/admin/update_min_asset_amount_in.rs:20-32, programs/bc-solana-jump-router-sc/src/instructions/admin/update_min_liquidity_amount_in.rs:20-32). These thresholds apply to both standard and headless swaps at `swap_accounts.rs:263-265` and `swap_accounts.rs:448-450`.

Impact: An admin can accidentally set `max_positive_slippage_mbps` so low that standard favorable-price swaps revert, or so high that the standard path's positive-slippage collar is effectively disabled. Separately, an admin can set either minimum input threshold above realistic trade sizes, DoSing the affected direction for both standard and headless swaps until corrected. Recovery is possible through admin updates, so the impact is operational rather than permanent.

Recommended Mitigation: Define and enforce acceptable bounds for `max_positive_slippage_mbps`, `min_asset_amount_in`, and `min_liquidity_amount_in` at initialization and in their setters. If exact bounds are business-configurable, document them and add defensive upper limits that prevent accidental all-swap DoS.

Securitize: Acknowledged; We don't need those limits, we understand the risk of an operator setting wrong values, but at the same time we can fix it easily. In the headless flow NBBO is not used at all, so the max positive slippage value will not be used because there is no reference price to compare against.

7.2.3 `UpdateJumpPoolConfig::update_jump_pool_config` repoints mints but does not reset `min_asset_amount_in`, `min_liquidity_amount_in`

Description: `update_jump_pool_config_handler` replaces `jump_router_state.jump_pool_config` with the new config, which may carry different `asset_mint` and `liquidity_mint` values with different decimal scales (programs/bc-solana-jump-router-sc/src/instructions/admin/update_jump_pool_config.rs:88-99). The mint accounts used by the update are checked only for key and token-program consistency (update_jump_pool_config.rs:27-47).

`min_asset_amount_in` and `min_liquidity_amount_in` are stored in `JumpRouterState` as native token-unit thresholds denominated in the respective mint's decimals. The handler does not reset or rescale those fields after changing the pool config.

If the new mints have a different decimal scale from the previous mints, the stored thresholds become mis-denominated. For example, if `min_asset_amount_in` was set to 1_000_000 for a 6-decimal asset mint and the config is updated to a 9-decimal asset mint, the threshold now represents 0.001 tokens rather than 1 token. The reverse migration can make the threshold unexpectedly large and block swaps until corrected.

Impact: After a pool config update that changes mint decimals, the minimum trade thresholds silently apply the wrong effective minimum. The issue is admin-recoverable by calling the threshold setters, but there is a misconfiguration window between the pool update and the corrective threshold updates.

Recommended Mitigation: Reset `min_asset_amount_in` and `min_liquidity_amount_in` when the configured mints change, or require the new thresholds to be supplied in the same instruction/transaction. If the current behavior is intended, document the required operational sequence: update pool config, then immediately update both minimum thresholds for the new mint units.

Securitize: Fixed in [90b345b](#).

Cyfrin: Verified.

7.2.4 `calculate_jump_price` returns a liquidity-over-asset ratio while three unit tests assert the inverse

Description: `calculate_jump_price` computes a WAD-scaled liquidity-per-asset price:

```
price = normalized_liquidity_amount * 1e18 / normalized_asset_amount
```

The implementation is at programs/bc-solana-jump-router-sc/src/utils/swap_utils.rs:26-59. The swap logic compares this value directly against the backend-supplied `nbbo_price` when the standard operator path provides one.

Several unit tests in `#[cfg(test)]` still assert the inverse convention:

- `calculates_price_with_equal_decimals` passes (`asset=2_000_000`, `liquidity=1_000_000`) and expects `2e18`, but the liquidity-per-asset result is `0.5e18`.
- `supports_price_above_u64_max` passes (`asset=20`, `liquidity=1`) and expects `20e18`, but the liquidity-per-asset result is `0.05e18`.
- `supports_price_quotient_above_u128_max` comments that price scales with `asset_amount`, but the implemented formula divides by the normalized asset amount.

These tests are at programs/bc-solana-jump-router-sc/src/utils/swap_utils.rs:91-129.

Impact: Production code currently uses the liquidity-per-asset convention, but the tests document and assert the inverse for multiple cases. This creates maintenance risk: a future change could flip the production formula to satisfy the tests, or backend/event consumers could infer the wrong NBBO convention from the test suite.

Recommended Mitigation: Update the affected tests to assert the liquidity-per-asset formula, and add a short comment or function doc stating that `calculate_jump_price` returns WAD-scaled liquidity units per asset unit.

Securitize: Fixed in [8dd4999](#).

Cyfrin: Verified.

7.2.5 JumpRouterCounter uses `init_if_needed` with a fixed singleton seed

Description: The `jump_router_counter` PDA is initialized with `init_if_needed` using the fixed seed `[JUMP_ROUTER_COUNTER_SEED]` (`programs/bc-solana-jump-router-sc/src/instructions/admin/initialize.rs:88-95`). Each call to `initialize_handler` reads the current counter value, creates a new router state at `[JUMP_ROUTER_STATE_SEED, counter.to_le_bytes()]`, then increments the counter (`initialize.rs:131-154`).

If an operator pre-derives and pre-signs an initialize transaction when the counter is `N`, and another allowed initializer lands an initialize transaction first, the counter advances to `N + 1`. The pre-signed transaction still targets the state PDA for counter `N`, which now exists, so the `init` constraint fails with an `account-already-in-use` error.

Impact: The pre-signed initialize transaction becomes invalid and must be rebuilt with the new counter value. Recovery is straightforward, so no funds are at risk and no permanent state is corrupted. The impact is an unexpected initialization failure for offline/pre-signed initialization flows.

Recommended Mitigation: Document that initialize transactions must derive the target router state from the live counter immediately before signing. If the program does not need multiple router instances, remove the counter and use a fixed singleton state PDA.

Securitize: Fixed in [8cb8e4d](#).

Cyfrin: Verified.

7.2.6 MbpsFeeManager doc comments mislabel milli-basis-points as basis points

Description: `MbpsFeeManager` operates in milli-basis-points with denominator `100_000`, but its comments call the units "bps" and "Basis points". A basis point denominator would be `10_000`; this implementation uses `100_000`, so one unit is a milli-basis-point.

Current comments:

```
programs/bc-solana-jump-router-sc/src/states/fee_manager/mpbs_fee_manager.rs
18:  /// Mbps-based fee manager (basis points)
19:  pub struct MbpsFeeManager {
20:      /// Fee numerator (bps)
21:      pub numerator: u32,
27:      /// Basis points denominator (100,000 = 0.001%)
28:      pub const DENOMINATOR: u32 = 100_000;
```

Recommended Mitigation: Replace "bps" / "Basis points" with "milli-bps" / "milli-basis-points" in the comments.

Securitize: Fixed in [8eb6e94](#).

Cyfrin: Verified.

7.2.7 Source filename `mpbs_fee_manager.rs` has transposed letters

Description: The file defining `MbpsFeeManager` is named `mpbs_fee_manager.rs`, but the type and terminology use `Mbps` for milli-basis-points. The transposed filename makes the module harder to find and can propagate into future imports.

Current locations:

```
programs/bc-solana-jump-router-sc/src/states/fee_manager/mpbs_fee_manager.rs
19:  pub struct MbpsFeeManager {

programs/bc-solana-jump-router-sc/src/states/fee_manager/mod.rs
```

```
2: mod mpbs_fee_manager;
5: pub use mpbs_fee_manager::*;
```

Recommended Mitigation: Rename the file to `mbps_fee_manager.rs` and update the module declarations in `programs/bc-solana-jump-router-sc/src/states/fee_manager/mod.rs`.

Securitize: Fixed in [194d314](#).

Cyfrin: Verified.

7.2.8 Standard `execute_swap` accepts an empty regulatory `external_id`

Description: The standard backend/operator-gated `execute_swap` instruction accepts `external_id`: String as an instruction argument but never validates that it is non-empty or otherwise well-formed:

- `programs/bc-solana-jump-router-sc/src/lib.rs:94-112`
- `programs/bc-solana-jump-router-sc/src/instructions/operator/execute_swap.rs:20-55`

The value is forwarded unchanged into the shared swap implementation and emitted as the only business identifier in the `PstRamp` event on both directions (`swap_accounts.rs:410-420`, `swap_accounts.rs:600-610`, `schema at events.rs:98-108`).

The headless path intentionally uses an empty external id:

```
pub const SWAP_HEADLESS_EXTERNAL_ID: &str = "";
```

That matches what was stated in the kickoff meeting, which describe headless swaps as deliberately removing reporting features such as the external IDs used for regulatory tracking. The gap is that the standard MBBO/backend-authorized path has the same capability: an operator can submit `execute_swap` with `external_id == ""`, and the router will execute the swap and emit an authoritative `PstRamp` record with a blank regulatory identifier — without going through the explicit headless path.

Recommended Mitigation: Reject blank standard external IDs before any swap execution:

```
require!(external_id.trim().is_empty(), JumpRouterError::InvalidExternalId);
require!(external_id.len() <= MAX_EXTERNAL_ID_LEN, JumpRouterError::InvalidExternalId);
```

Securitize: Fixed in [35220bb](#).

Cyfrin: Verified.

7.2.9 `RedemptionCompleted` emits gross liquidity output rather than the user's net received amount

Description: The `RedemptionCompleted` event currently reports the gross `received_liquidity_amount` instead of the net `liquidity_amount_net` actually transferred to the user after fees. This does not affect fund safety, but it weakens event accuracy and can mislead off-chain accounting, analytics, and monitoring systems.

In the `AssetForLiquidity` flow, the program first computes the gross amount returned by the pool, then derives the net user amount by subtracting protocol and optional external fees:

```
let liquidity_fee_amount = jump_router_state
    .fee_manager
    .calculate_fee(quoted_liquidity_amount)?;

let liquidity_external_fee_amount = match external_fee_manager {
    Some(manager) => manager.calculate_fee(quoted_liquidity_amount)?,
    None => 0,
};

let liquidity_amount_net = quoted_liquidity_amount
    .checked_sub(liquidity_fee_amount)
    .and_then(|x| x.checked_sub(liquidity_external_fee_amount))
```

```
.ok_or(JumpRouterError::MathOverflow)?;
```

The user then receives `liquidity_amount_net`:

```
crate::transfer_tokens!(
    &swap_accounts.liquidity_router_vault,
    &swap_accounts.liquidity_user_ata,
    &swap_accounts.jump_router_authority,
    &swap_accounts.liquidity_program,
    &swap_accounts.liquidity_mint,
    liquidity_amount_net,
    swap_accounts.liquidity_mint.decimals,
    router_authority_signer
);
```

However, the emitted event reports the gross amount instead:

```
emit_cpi!(crate::events::RedemptionCompleted {
    off_ramp: jump_router_state.key(),
    redeemer: swap_accounts.user.key(),
    asset_amount: amount_in,
    liquidity_amount: received_liquidity_amount,
    fee_amount: liquidity_fee_amount,
    rate: denormalized_jump_price,
});
```

As a result, consumers relying only on the event may infer that the user received more liquidity than was actually credited to their account.

Impact: This issue is limited to observability and reporting quality. It can cause discrepancies in indexers, reconciliation pipelines, dashboards, or downstream integrations that assume the event reflects the user's net settlement amount.

Recommended Mitigation: Consider emitting the net amount delivered to the user, or extend the event to include both gross and net values explicitly. This preserves full transparency while allowing off-chain consumers to reason about settlement and fees without reconstructing them manually.

Securitize: Fixed in [bc33a93](#).

Cyfrin: Verified.

7.2.10 Use a two-step admin transfer flow to reduce the risk of irreversible governance mistakes

Description: The router updates the admin in a single transaction. While this is not an exploitable security issue by itself, a one-step transfer increases governance and operational risk because an incorrect address becomes authoritative immediately, with no acceptance step by the new admin.

The current admin transfer flow directly overwrites `jump_router_state.admin` after validating that the new key is non-default and different from the current admin:

```
pub fn change_admin_handler(ctx: &mut Context<ChangeAdmin>, new_admin: Pubkey) -> Result<()> {
    let jump_router_state = &mut ctx.accounts.jump_router_state;

    require!(
        new_admin != Pubkey::default(),
        JumpRouterError::DefaultPubkeyNotAllowed
    );

    require!(
        jump_router_state.admin != new_admin,
        JumpRouterError::NoChange
    );
}
```



```

    jump_router_state.admin = new_admin;

    emit!(crate::events::AdminChanged {
        jump_router: jump_router_state.key(),
        admin: ctx.accounts.admin.key(),
        new_admin,
    });

    Ok(())
}

```

Because the transfer completes in one step, any mistake in the destination address immediately changes control of the router. A two-step pattern is generally preferred for privileged role transfers because it requires the nominated admin to explicitly accept the role before the change becomes effective.

Impact: It increases the likelihood of governance mistakes, especially during key rotation, incident response, or operational handovers.

Recommended Mitigation: Adopt a two-step admin transfer flow.

Securitize: Acknowledged.

7.2.11 Router admin can repoint an existing router to an unrelated Jump pool

Description: `update_jump_pool_config_handler` allows the router admin to replace the full `JumpPoolConfig`, including the underlying pool address, as long as the new config validates against the provided pool state.

There is no invariant that the new pool address must match the router's existing pool address. As a result, a router that was initialized for one `BisonFi` pool can later be repointed to another valid pool, potentially one that was expected to be served by a different router.

Impact: This creates configuration ambiguity and weakens router-to-pool isolation assumptions. Operators, users, indexers, or off-chain systems may treat a router as the canonical gateway for a specific pool, but the admin can change that router to execute swaps against a different pool without creating a new router.

Recommended Mitigation: If routers are intended to be bound to a single pool, make the pool address immutable after initialization. In `update_jump_pool_config_handler`, require the new config address to equal the current config address:

```

require!(
    jump_pool_config.address == jump_router_state.jump_pool_config.address,
    JumpRouterError::InvalidJumpPoolConfig
);

```

Securitize: Acknowledged.

7.2.12 Fee collector default pubkey is not rejected while initialization

Description: During router initialization, the program validates that `fee_collector_wallet` matches the wallet configured in `fee_manager`, but it does not explicitly reject `Pubkey::default()`.

However, `UpdateFeeManager` applies a stricter check and correctly rejects the default pubkey:

```

fee_collector_wallet.key() == fee_manager.fee_collector_wallet()
&& fee_collector_wallet.key() != Pubkey::default()

```

This creates inconsistent validation between initialization and later fee-manager updates. If a router is initialized with the default pubkey as the fee collector wallet, protocol fees may be transferred to the associated token account owned by the default pubkey.

This is mainly a defense-in-depth/configuration issue because initialization is permissioned, but it can still cause permanent loss of fee revenue if misconfigured.

Recommended Mitigation: Apply the same non-default wallet validation during initialization as well:

```
#[account(  
    constraint = fee_collector_wallet.key() == fee_manager.fee_collector_wallet()  
    @@ fee_collector_wallet.key() != Pubkey::default()  
    @ JumpRouterError::InvalidFeeCollector,  
)]  
pub fee_collector_wallet: UncheckedAccount<'info>,
```

Also consider enforcing this inside `FeeManager::validate()` so all fee-manager entry points share the same invariant.

Securitize: Fixed in [64086a4](#).

Cyfrin: Verified.

7.2.13 Combined internal and external fees are not capped

Description: The program validates each fee manager independently, but it does not enforce a maximum total fee across both the internal router fee and the optional external fee. Each `FeeManager` is individually validated against `MAX_FEE_NUMERATOR`, which currently allows up to 50% per fee manager. This means a swap can apply two individually valid fee managers whose combined fee exceeds the intended maximum fee limit.

Impact: The effective fee charged to users can be higher than the protocol's intended maximum. In the worst case, most or all of the fee-denominated liquidity amount may be consumed by fees. Although `min_amount_out` can protect users when configured correctly, the protocol itself does not enforce the combined-fee invariant.

Recommended Mitigation: Validate the combined internal and external fee rate before executing the swap. For example, expose the fee numerator from each `FeeManager`, or add a method to validate combined fee managers:

```
require!(  
    internal_fee_numerator + external_fee_numerator <= MbpsFeeManager::MAX_FEE_NUMERATOR,  
    JumpRouterError::MaxFeeExceeded  
);
```

Securitize: Acknowledged.

7.3 Gas Optimization

7.3.1 Recomputed power-of-ten divisor in repeated per-swap price denormalization

Description: The shared swap implementation repeatedly calls `denormalize_price_to_decimals` with the same decimal value during event emission.

On the standard operator-gated path, `nbbo_price_opt` is `Some`, so both NBBO and JUMP prices are denormalized in the buy direction (`programs/bc-solana-jump-router-sc/src/instructions/swap_accounts.rs:393-404`) and in the sell direction (`swap_accounts.rs:583-594`). Each call recomputes `10u128.pow(scale_diff)` inside `denormalize_price_to_decimals` (`programs/bc-solana-jump-router-sc/src/utis/swap_utils.rs:77-84`).

On the headless path, `nbbo_price_opt` is `None`, so only the JUMP price is denormalized. The repeated computation therefore applies to standard swaps, while headless swaps pay the cost once per swap.

Impact: This is a small per-swap compute inefficiency on standard swaps. It has no correctness impact.

Recommended Mitigation: If the optimization is worth the added code, compute the scale divisor once and use it for both NBBO and JUMP price denormalization. Alternatively, leave the current implementation for readability.

Securitize: Acknowledged; The impact is negligible.